

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability.

Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures.

Recommend a robust integrity verification mechanism and justify its effectiveness.

1. Weak Password Hashing in Web Application

Analysis of the Vulnerability

The web application stores user passwords using **unsalted SHA-1 hashes**. This creates a serious security weakness because SHA-1 is a fast general-purpose hashing algorithm and is not designed specifically for password storage. After the database was breached, attackers could use precomputed password tables and high-speed cracking tools to recover many of the original passwords quickly.

The main problem is the **absence of salting**. A salt is a unique random value added to each password before hashing. Without a salt, two users with the same password produce exactly the same hash. Attackers can therefore calculate or obtain precomputed tables of common password hashes and compare them directly with the stolen hashes. This makes large-scale password cracking much easier.

The use of **SHA-1 further increases the risk** because it is computationally fast. Although speed is useful for normal hashing applications, password hashing should deliberately be slow and computationally expensive so that attackers cannot test millions or billions of password guesses quickly. SHA-1 is also considered obsolete for modern security applications.

Recommended Secure Password Hashing Strategy

The application should replace SHA-1 with a **password-specific hashing algorithm such as Argon2id**. Argon2id is designed to resist modern password-cracking attacks by requiring significant computational resources and memory.

A secure password storage process should be:

1. Generate a unique, cryptographically secure random salt for every password.
2. Combine the password with the salt.
3. Hash the password using Argon2id with appropriately configured time and memory costs.
4. Store the resulting password hash together with the salt and hashing parameters.
5. During login, apply the same hashing process to the entered password and compare the result with the stored hash.

For example:

Password + Unique Salt → Argon2id → Stored Password Hash

The salt does not need to be kept secret. Its purpose is to ensure that identical passwords produce different hashes and to make precomputed rainbow-table attacks ineffective.

Justification

Argon2id is a stronger choice because it is specifically designed for password hashing rather than general-purpose hashing. Its configurable memory and computational requirements make large-scale password cracking considerably more expensive for attackers. Using a unique salt for every

password additionally prevents attackers from efficiently reusing precomputed hashes across multiple accounts.

Therefore, the combination of **Argon2id + unique random salts + appropriate security parameters** provides substantially stronger protection than unsalted SHA-1 and reduces the likelihood of attackers recovering passwords even if the password database is compromised. This directly addresses both weaknesses identified in the case: the lack of salting and the inappropriate use of a fast, outdated hashing algorithm.

2. Digital Signature Compromise in Financial System

Analysis of the Security Impact

The financial institution uses **digital signatures** to approve transactions. In this case, an attacker gains access to a user's **private key** and uses it to create unauthorized transaction signatures. This compromises the security of the digital signature system because the attacker can generate signatures that appear to be legitimately created by the affected user.

The compromise directly affects **authentication**. A digital signature is used to verify that a transaction was signed using the corresponding private key. Since the attacker possesses the legitimate private key, the verification process may accept the attacker's unauthorized transactions as authentic.

It also affects **non-repudiation**. Normally, a valid digital signature provides evidence that a particular private key was used to authorize a transaction, making it difficult for the signer to deny having approved it. However, once the private key has been stolen, a valid signature could have been generated either by the legitimate user or by the attacker. Therefore, the institution can no longer reliably associate every signed transaction with the legitimate user's actions.

Mitigation Strategies

The financial institution should implement multiple measures to protect private keys:

1. **Hardware Security Modules (HSMs):**
Store sensitive private keys inside HSMs so that the keys cannot be easily extracted or copied. Cryptographic signing operations can be performed within the HSM.
2. **Strong Access Control:**
Restrict access to private-key operations using strong authentication, role-based access control, and the principle of least privilege.
3. **Multi-Factor Authentication (MFA):**
Require an additional authentication factor before approving sensitive financial transactions. This provides another security layer even if a private key or user credential is compromised.
4. **Key Rotation and Revocation:**
Private keys should have controlled lifetimes and be replaced periodically. If a key is suspected of being compromised, the corresponding certificate should be revoked immediately so that it is no longer trusted.

5. **Transaction Monitoring:**

Monitor signed transactions for unusual patterns, such as unexpected transaction amounts, destinations, or frequencies. Suspicious transactions can then be blocked or subjected to additional verification.

6. **Secure Key Backup:**

Backup copies of private keys must also be protected using secure cryptographic storage. An insecure backup could simply give the attacker another copy of the same key.

Conclusion

A compromised private key seriously undermines both **authentication and non-repudiation**, because an attacker can create valid-looking signatures on behalf of the legitimate user. The financial institution should therefore protect private keys using mechanisms such as **HSMs, strong access controls, MFA, key rotation, certificate revocation, and transaction monitoring**. These measures reduce the probability of unauthorized signing and help maintain trust in the digital signature system.

3. Kerberos Authentication Failure

Analysis of the Problem

The distributed system uses **Kerberos** for authentication, but frequent authentication failures occur because of **time synchronization problems between servers**. Kerberos relies heavily on accurate timestamps to prevent attackers from reusing old authentication messages.

During Kerberos authentication, tickets and authentication messages contain timestamps. The receiving system checks whether the timestamp is within an acceptable time difference, known as the **clock skew**. If the clocks of the client, authentication server, and other participating systems are significantly different, a legitimate authentication request may be considered invalid.

For example, if a client generates an authentication request with a timestamp that appears too old or too far in the future according to the server's clock, Kerberos may reject the request. This results in repeated authentication failures even though the user's credentials are correct.

Security Importance of Time Synchronization

Time synchronization is not only required for reliable operation but also contributes to **security**. Kerberos uses timestamps to provide protection against **replay attacks**, where an attacker attempts to reuse a previously captured authentication message.

If timestamp checking were disabled or made excessively permissive simply to avoid authentication failures, an attacker could potentially reuse captured authentication information. Therefore, the system must maintain accurate clocks while keeping an appropriate clock-skew limit.

Recommended Solutions

1. **Use Network Time Protocol (NTP):**
Configure all Kerberos clients and servers to synchronize their clocks with trusted NTP servers.
2. **Use a Reliable Time Source:**
The Kerberos Key Distribution Center (KDC), clients, and application servers should use consistent and reliable time sources.
3. **Monitor Clock Drift:**
Implement monitoring to detect systems whose clocks are drifting significantly from the expected time.
4. **Configure an Appropriate Clock-Skew Limit:**
The allowed time difference should be configured carefully. Increasing it excessively may reduce protection against replay attacks.
5. **Synchronize All Kerberos Components:**
The authentication server, ticket-granting server, clients, and participating application servers must maintain consistent system time.
6. **Generate Alerts for Synchronization Failures:**
Administrators should receive alerts when a system becomes significantly out of synchronization so that the problem can be corrected before authentication failures become widespread.

Conclusion

Kerberos depends on accurate time synchronization because timestamps are used to validate authentication messages and help prevent replay attacks. Large differences between system clocks can therefore cause legitimate authentication requests to fail. The organization should use **reliable NTP synchronization, clock-drift monitoring, appropriate clock-skew configuration, and alerts for synchronization problems**. These measures improve authentication reliability while preserving the security benefits provided by Kerberos timestamps.

4. Certificate Authority Breach in PKI System

Analysis of the Security Impact

A **Certificate Authority (CA)** is a trusted entity in a Public Key Infrastructure (PKI) that issues digital certificates, including **X.509 certificates**, to establish the identity of websites and other entities. In this case, the CA is compromised and attackers use it to issue fake certificates for trusted domains.

The major problem is that users and systems normally trust certificates issued by a legitimate CA. If attackers obtain the ability to issue fraudulent certificates, they can create a certificate that appears to be valid for a trusted domain. As a result, users may unknowingly establish secure connections with an attacker instead of the legitimate server.

This can enable **man-in-the-middle (MITM) attacks**. The attacker can potentially intercept encrypted communication, impersonate the legitimate domain, and attempt to steal sensitive information such as login credentials or financial data.

The breach therefore damages the **chain of trust** within the PKI system. Even though encryption may still be functioning correctly, users can no longer be confident that the public key in a certificate actually belongs to the intended domain.

Detection Mechanisms

1. **Certificate Transparency (CT):**
Monitor publicly logged certificates to identify unexpected or fraudulent certificates issued for trusted domains.
2. **Certificate Validation:**
Systems should verify the certificate chain, validity period, issuer, domain name, and other certificate properties before establishing trust.
3. **Certificate Revocation Checking:**
Use mechanisms such as **Certificate Revocation Lists (CRLs)** and **Online Certificate Status Protocol (OCSP)** to determine whether a certificate has been revoked.
4. **Security Monitoring:**
Organizations should monitor their domains for unauthorized certificate issuance and investigate suspicious certificates immediately.

Mitigation Strategies

1. **Revoke Fraudulent Certificates:**
Any certificates issued by the compromised CA for affected domains should be identified and revoked.
2. **Secure the CA Infrastructure:**
CA private keys and certificate-issuance systems should be protected using strong access controls and secure hardware where appropriate.
3. **Use Strong Authentication and Access Control:**
Administrative access to CA systems should require strong authentication and should be restricted according to the principle of least privilege.
4. **Key Management and Rotation:**
Compromised CA keys should be replaced and securely managed to prevent further unauthorized certificate issuance.
5. **Continuous Certificate Monitoring:**
Domain owners should continuously monitor certificate logs and Certificate Transparency records for certificates that they did not request.

Conclusion

A compromised CA can seriously damage trust in a PKI environment because attackers may obtain fraudulent X.509 certificates for trusted domains. This can enable impersonation and man-in-the-middle attacks and undermine secure communication. **Certificate Transparency, certificate revocation, secure CA infrastructure, strong access control, key management, and continuous monitoring** can help detect and mitigate such attacks. These mechanisms help restore confidence in the certificate-based trust model on which PKI depends.

5. Integrity Verification in Cloud Data Storage

Analysis of the Problem

The cloud service provider uses **hashing to verify file integrity**, but users report data tampering because the hash validation process has been implemented incorrectly. Hashing is intended to produce a fixed-length value that represents the contents of a file. If the file changes, its hash should also change, allowing the system to detect unauthorized modification.

Incorrect implementation can cause integrity verification to fail. For example, if the system does not correctly calculate the hash after storing or retrieving a file, it may compare the wrong hash values and incorrectly consider modified data to be valid. Similarly, using a weak hashing algorithm or failing to securely protect the expected hash value can allow tampered data to go undetected.

Another important issue is **how the reference hash is stored and validated**. If an attacker can modify both the file and its stored hash value, simply recalculating the hash will not detect the tampering because the attacker can replace the original hash with a new one corresponding to the modified file.

Recommended Integrity Verification Mechanism

A robust solution should use a **cryptographically secure hash algorithm such as SHA-256** together with secure storage and validation of the expected hash value.

The process can be implemented as follows:

1. Calculate the SHA-256 hash of the original file.
2. Store the original hash in a secure location separate from the user-controlled file.
3. When the file is retrieved, calculate its SHA-256 hash again.
4. Compare the newly calculated hash with the securely stored original hash.
5. If the values match, the file is considered unchanged.
6. If the values differ, the system should flag the file as potentially modified or corrupted.

For stronger protection against an attacker who can modify stored metadata, the system can use a **keyed integrity mechanism such as HMAC-SHA-256**. The secret key is kept separately from the stored files, making it much harder for an attacker to generate a valid integrity value for modified data.

Additional Security Measures

- Use a strong cryptographic hash such as **SHA-256** rather than outdated or weak hashing algorithms.
- Store reference hashes or authentication keys in a **secure location** that ordinary users cannot modify.
- Apply access controls so that unauthorized users cannot replace integrity information.
- Perform integrity checks whenever files are uploaded, stored, or retrieved.
- Maintain logs of integrity failures so that suspicious modifications can be investigated.

- For highly sensitive cloud data, use authenticated integrity mechanisms such as **HMAC** rather than relying only on an unprotected hash.

Conclusion

Incorrect hashing and validation practices can allow modified cloud files to pass integrity checks, especially when the reference hash is not securely protected. A robust solution should use **SHA-256 with secure reference-hash storage**, or preferably **HMAC-SHA-256** when protection against intentional tampering is required. Regular validation, access control, and monitoring further strengthen the system and help ensure that unauthorized changes to cloud-stored data are detected.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand